

ubidots

Devices

Data

Users

Apps

PROGRAMACIÓN DE SOC ESP32 EN ANSI C Y APLICACIONES IOT



Temperatura (°C) - Nodo 02

25.60

Humedad - Nodo 02

Switch Control1 - Nodo3

n3_button
(red_lora_esp32)



On

Switch Control2 - Nodo3

n3_button2
(red_lora_esp32)



On



```
metodo POST");
```

```
client, HTTP_METHOD_POST);
```

```
ESP_LOGE(TAG, "Error en la configuracion de metodo HTTP POST");
```

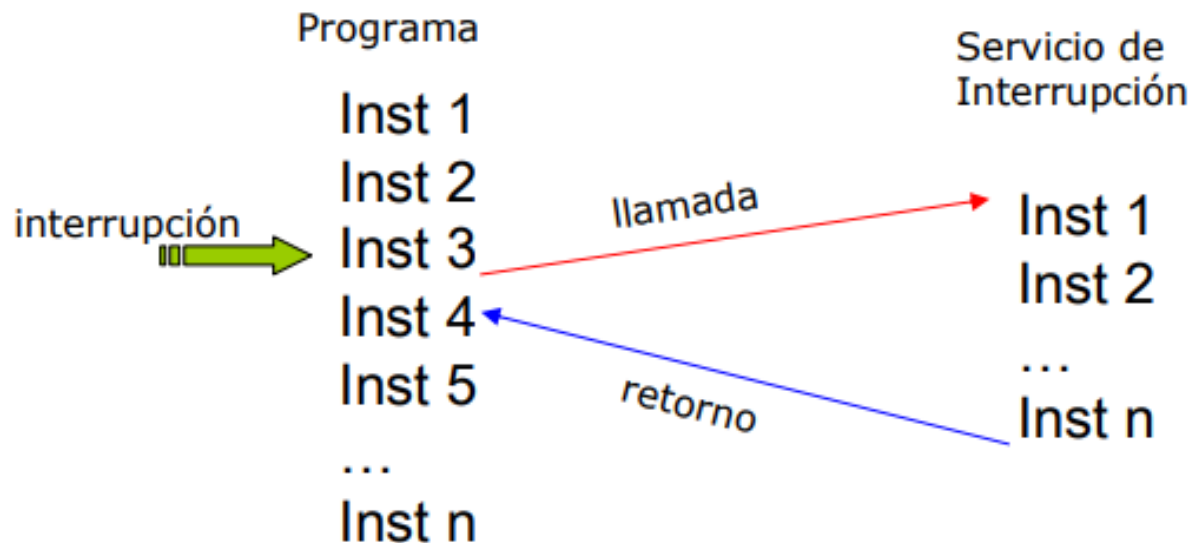
```
ret = esp_http_client_set_url(http_client, "http://industrial.api.ubidots.com/api/v1.6/devices/");
```

```
ESP_LOGE(TAG, "Error en la configuracion de metodo HTTP POST");
```

¿Qué es una interrupción?

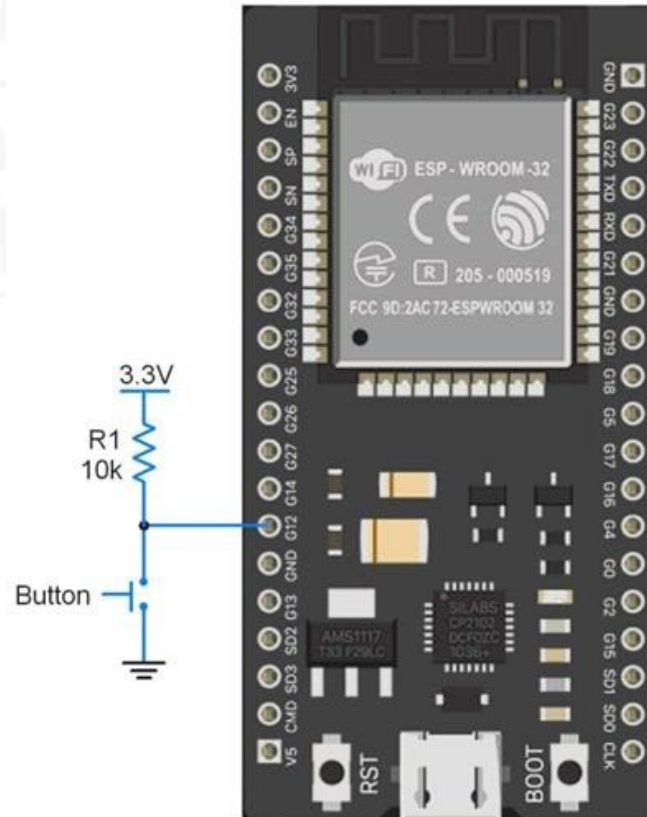
Las Interrupciones son eventos especiales que se generan interna o externamente en el procesador y que pausa momentáneamente la ejecución del programa para realizar una tarea breve, y después volver al punto donde se quedó ejecutando el programa.

Esto sirve para que el procesador no este siempre trabajando, por ejemplo, en checar el estado de una entrada constantemente, y solo poner atención cuando el evento deseado ocurre.



Técnica Polling: La CPU **consulta constantemente** el componente/dispositivo de hardware que emite una señal para ser atendida. Por lo general esta técnica degrada el rendimiento del sistema.

Interrupciones: La CPU no necesita consultar constantemente el componente/dispositivo de hardware ya que el sistema de interrupción le avisará cuando éste tenga una señal para ser atendida por la CPU y atender la petición.



Fuentes de interrupción periférica:

- ESP32 tiene 71 fuentes de interrupción periférica en total.
- 67 de 71 fuentes de interrupción periférica se pueden asignar a cualquiera de las CPU.
- Las cuatro fuentes de interrupción periféricas restantes son específicas de la CPU, dos por CPU.
- GPIO_INTERRUPT_PRO y GPIO_INTERRUPT_PRO_NMI solo se pueden asignar a PRO_CPU.
- GPIO_INTERRUPT_APP y GPIO_INTERRUPT_APP_NMI solo se pueden asignar a APP_CPU.
- Como resultado, PRO_CPU y APP_CPU se puede asignar hasta 69 fuentes de interrupción periféricas a cada una.

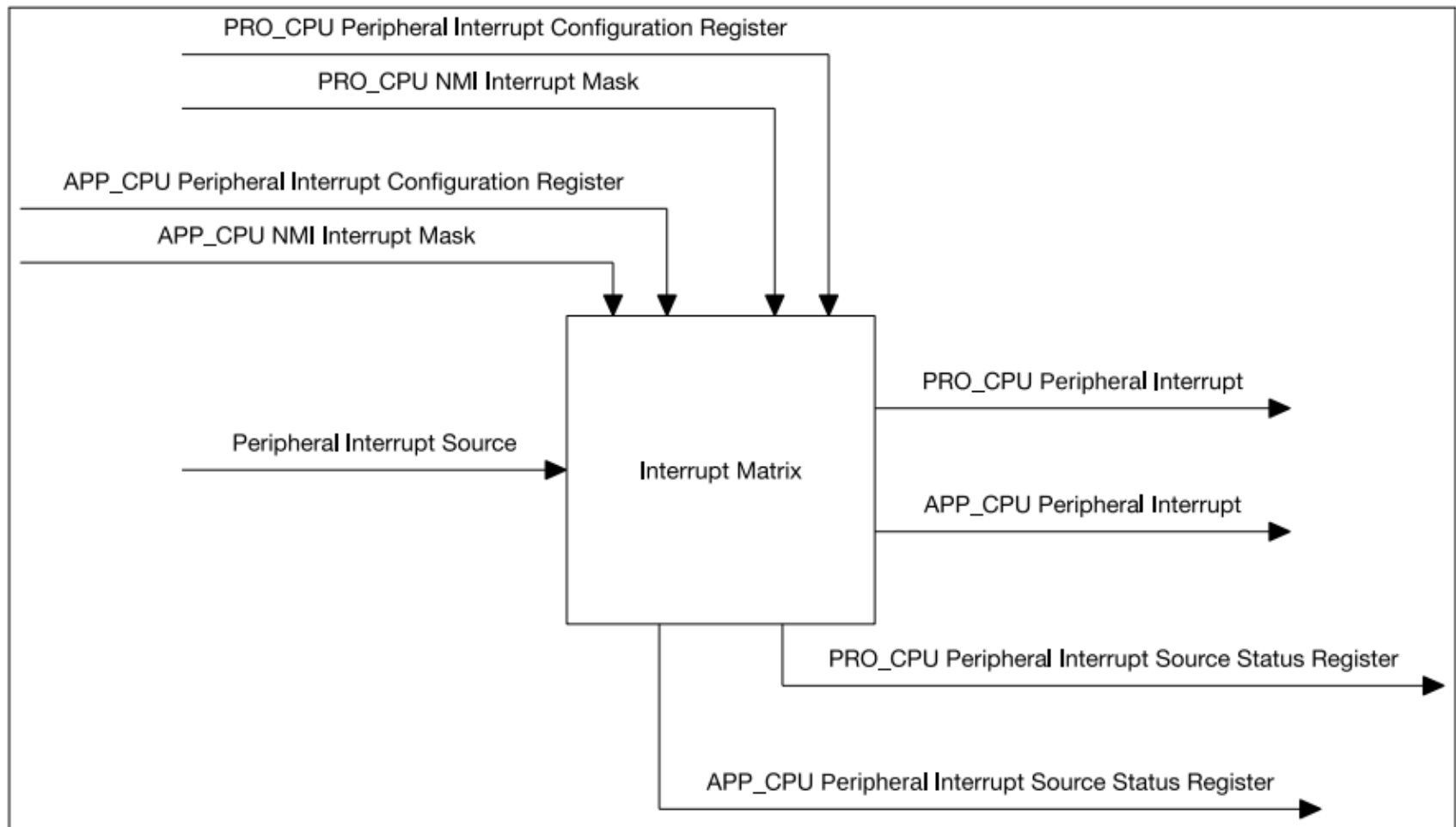
Interrupción de la CPU:

- El ESP32 ofrece hasta **32 ranuras** de interrupción para cada CPU (PRO y APP), de las cuales 26 son **interrupciones periféricas** y 6 son **interrupciones de los propios núcleos** de la CPU Xtensa.

Nota1: Revisar el documento [*esp32_technical_reference_manual_en*](#).

... Interrupt Matrix (INTERRUPT)

Estructura de la Matriz de interrupción

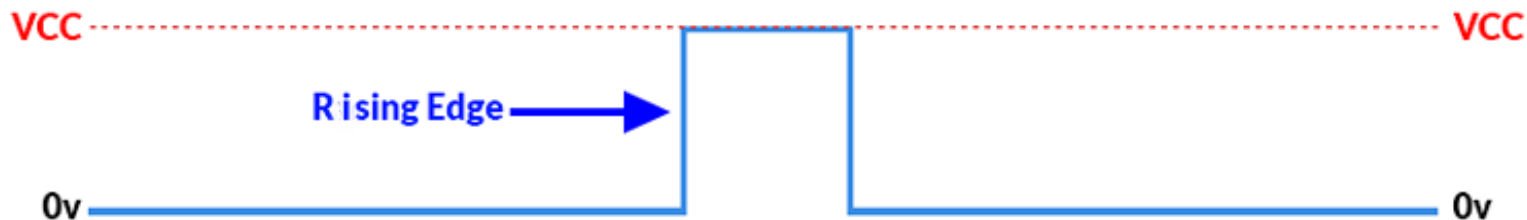


Una **interrupción GPIO** es una forma de interrupción externa en la que se produce una señal de activación externa cuando se presiona un botón, por ejemplo. Cuando se activa una interrupción, el procesador detiene la ejecución del programa principal. En este punto se llama a la **Rutina de Servicio de Interrupción (ISR)**. Luego, el procesador trabaja temporalmente en una tarea ISR, una vez que termina la tarea ISR regresa al programa principal.

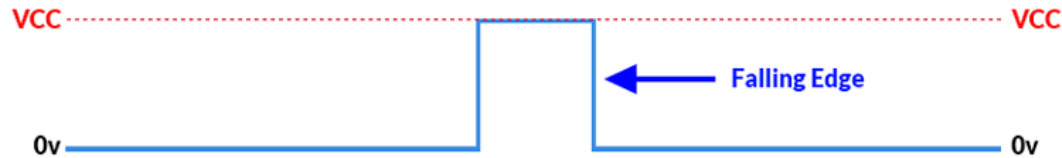
- En ESP32, **todos los pines GPIO** pueden ser configurados para funcionar como entradas de solicitud de interrupción (considerar que algunos pines están reservados en los DevKit).

Se admiten los siguientes tipos de interrupción en GPIO:

- **GPIO_INTR_POSEDGE** : rising Edge

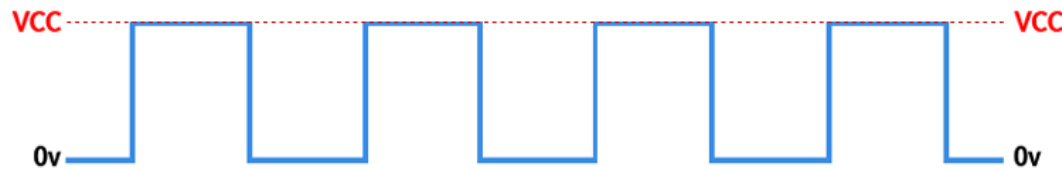


- **GPIO_INTR_NEGEDGE** : falling edge

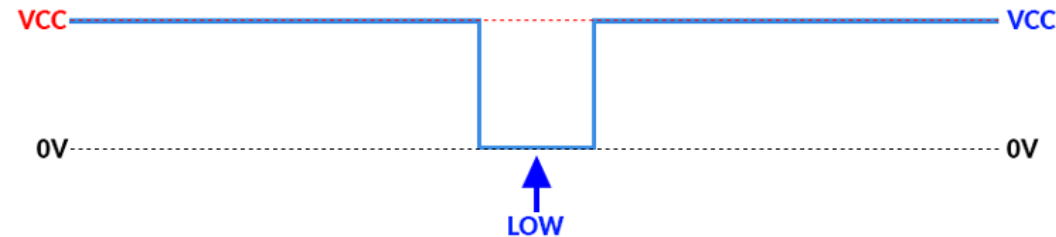


- **GPIO_INTR_ANYEDGE** : both rising and falling edge

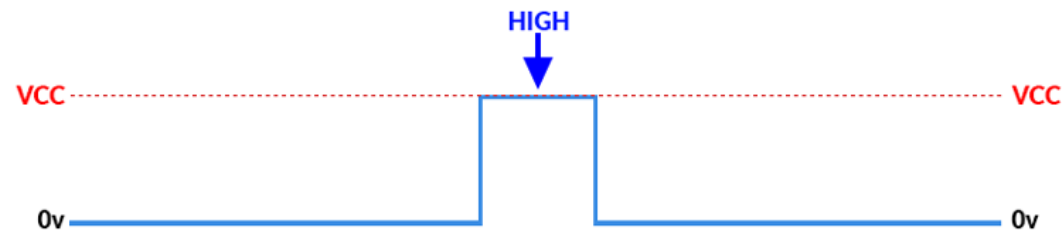
Interrupt whenever the pin changes value, from HIGH to LOW or LOW to HIGH



- **GPIO_INTR_LOW_LEVEL** : input low level trigger



- **GPIO_INTR_HIGH_LEVEL** : input high level trigger



La **Rutina de Servicio de Interrupción** se invoca cuando se produce una interrupción en cualquier pin GPIO. Su sintaxis es como la siguiente.

```
void IRAM_ATTR IsrPin (void *args)
{
    Sentencias;
}
```

Las ISR en ESP32 son tipos especiales de funciones que tienen algunas reglas únicas.

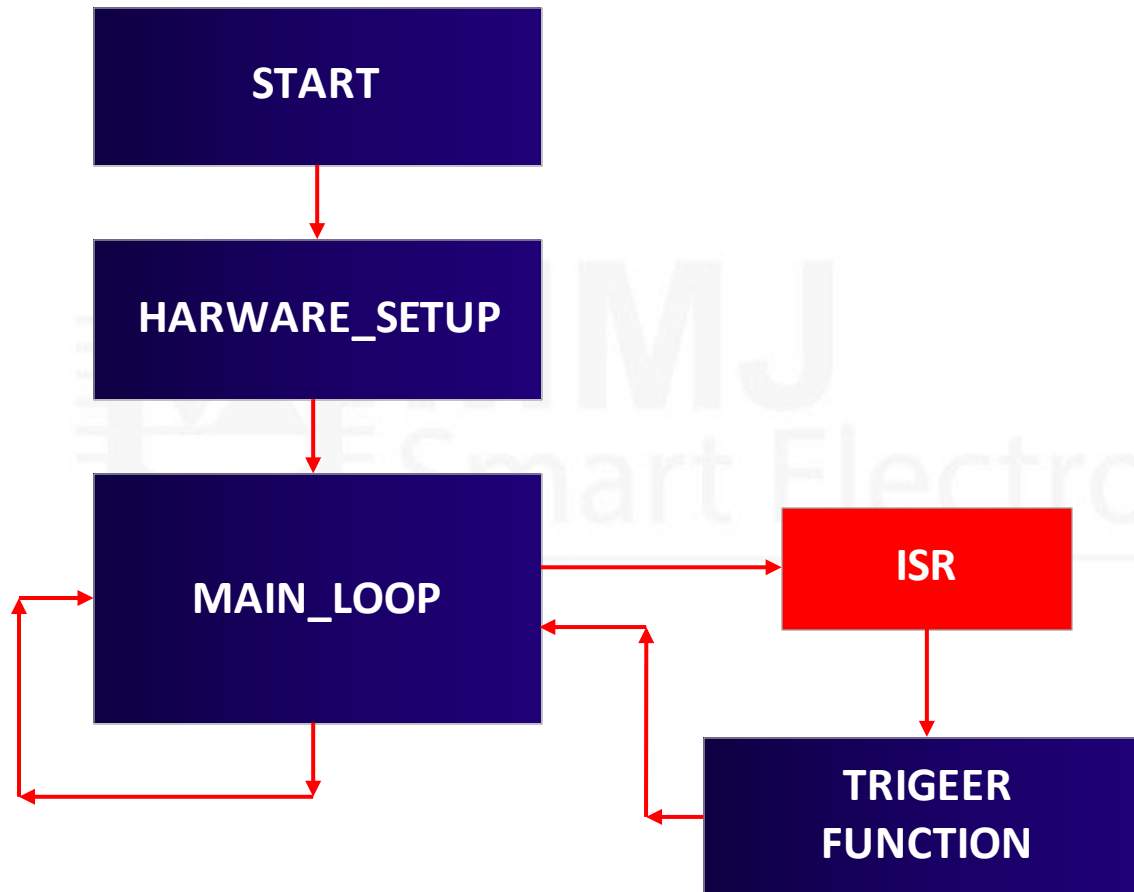
- La ISR debe tener un tiempo de ejecución lo más corto posible.
- Las ISR debe tener el atributo **IRAM_ATTR**, según la documentación de ESP32.

¿Qué es IRAM_ATTR?

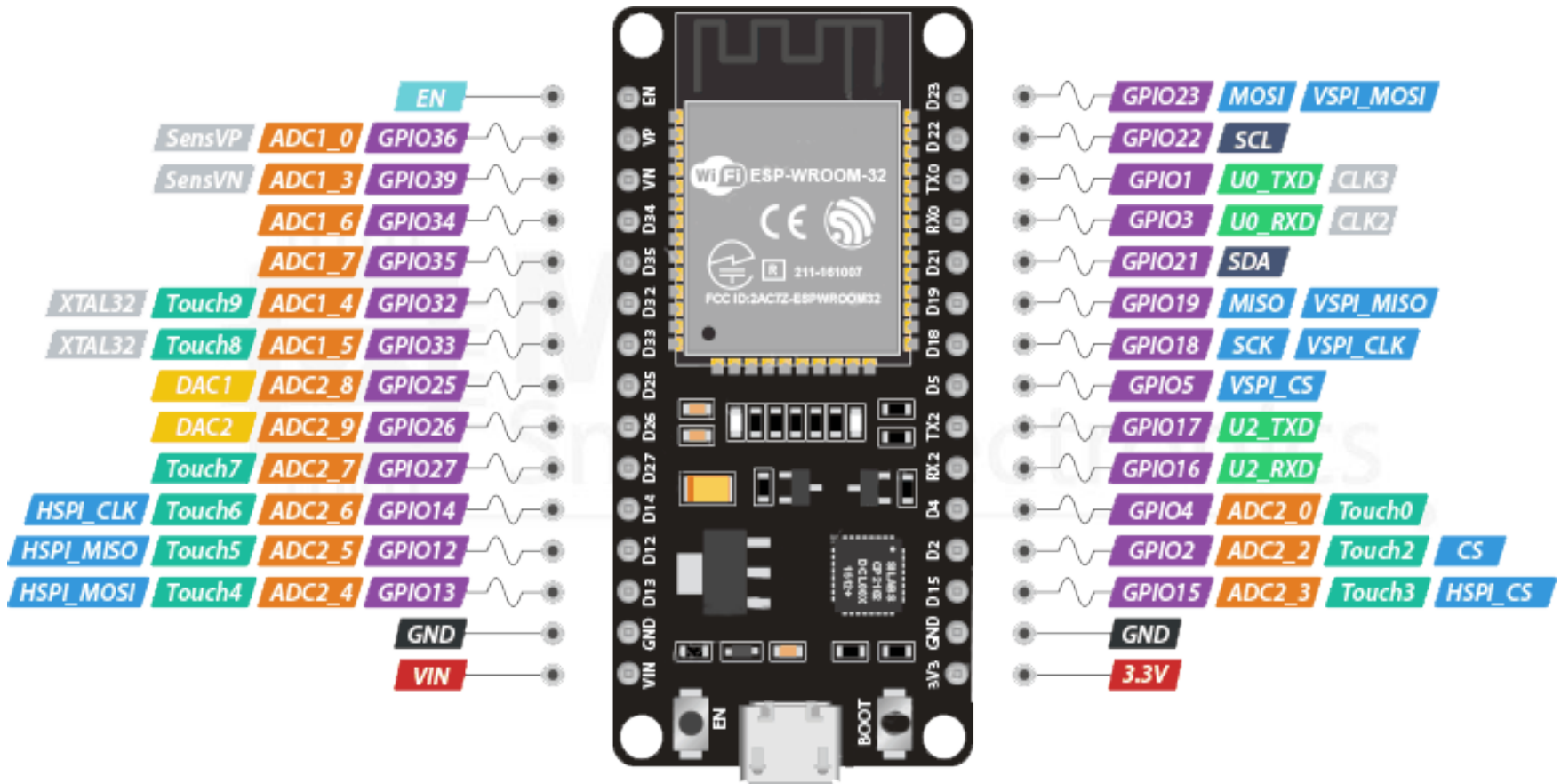
- Al marcar un bloque de código con el atributo **IRAM_ATTR** estamos declarando que el código compilado se colocará en la RAM interna (IRAM) del ESP32.
- De lo contrario, el código se coloca en la memoria Flash. Y el flash en el ESP32 es mucho más lenta que la RAM interna.

Nota: Se debe realizar configuración previa del **sdkconfig** en ESP-IDF.

Flujo de ejecución de una interrupción



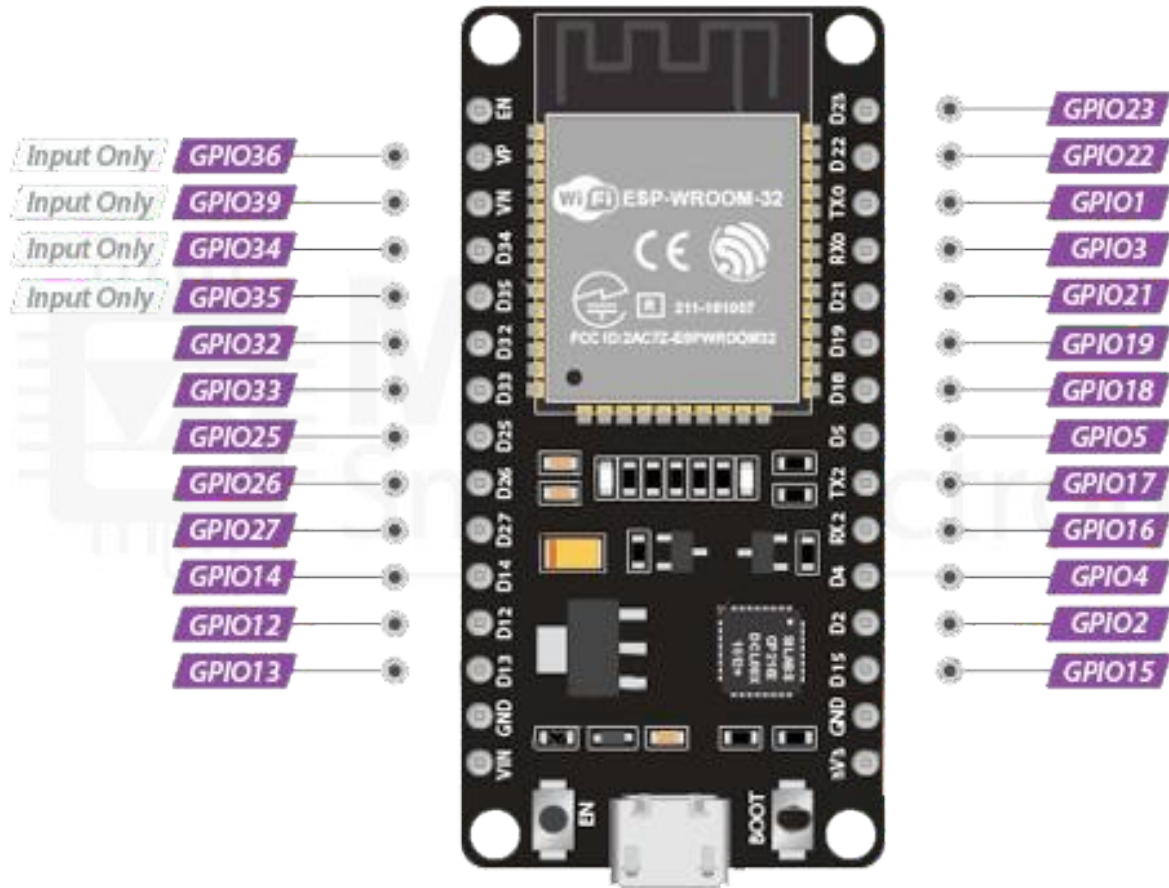
DEVKIT V1 ESP-WROOM-32



Notas al usar los GPIO de ESP32:

- **Strapping pin:** **GPIO0**, **GPIO2**, **GPIO5**, **GPIO12** (MTDI) y **GPIO15** (MTDO) son Strapping pin. Los Strapping pin se utilizan para poner el ESP32 en modo bootloader o flasheo. En la mayoría de las placas de desarrollo con USB/SERIAL incorporado, no necesita preocuparse por el estado de estos pines. La propia placa coloca los pines en el estado correcto antes del flasheo o cuando está en modo bootloader.
- <https://docs.espressif.com/projects/esptool/en/latest/esp32/advanced-topics/boot-mode-selection.html>
- **SPI0/1:** **GPIO6-11** y en algunos chips (**GPIO16-17**) generalmente se conectan al flash SPI y PSRAM integrados en el módulo y, por lo tanto, no deben usarse para otros fines.
- JTAG: **GPIO12-15** generalmente se usan para la depuración en línea.
- GPI: **GPIO34-39** solo se puede configurar como modo de entrada y no tienen funciones pull-up o pull-down habilitadas por software.
- **UART0:** TXD (**GPIO1**) y RXD (**GPIO3**) generalmente se usan para flashear y depurar.
- **ADC2:** los pines **GPIO ADC2** no se pueden usar cuando se usa Wi-Fi. Por lo tanto, debe considerar usar un **GPIO ADC1** en su lugar..

GPIO y GPI Pines



Nota: La placa de desarrollo ESP32 tiene 25 pines GPIO a los que se les pueden asignar diferentes funciones programando los registros apropiados.

Header File: [components/driver/gpio/include/driver/gpio.h](#)

Funciones Comunes:

```
esp_err_t gpio_config(const gpio_config_t *pGPIOConfig);
esp_err_t gpio_reset_pin(gpio_num_t gpio_num);
esp_err_t gpio_set_level(gpio_num_t gpio_num, uint32_t level);
int        gpio_get_level(gpio_num_t gpio_num);
esp_err_t gpio_set_direction(gpio_num_t gpio_num, gpio_mode_t mode);
esp_err_t gpio_set_pull_mode(gpio_num_t gpio_num, gpio_pull_mode_t pull);
esp_err_t gpio_wakeup_enable(gpio_num_t gpio_num, gpio_int_type_t intr_type);
esp_err_t gpio_wakeup_disable(gpio_num_t gpio_num);
esp_err_t gpio_pullup_dis(gpio_num_t gpio_num);
esp_err_t gpio_pulldown_en(gpio_num_t gpio_num);
esp_err_t gpio_pulldown_dis(gpio_num_t gpio_num);
esp_err_t gpio_set_intr_type(gpio_num_t gpio_num, gpio_int_type_t intr_type);
esp_err_t gpio_intr_enable(gpio_num_t gpio_num);
esp_err_t gpio_intr_disable(gpio_num_t gpio_num);
esp_err_t gpio_install_isr_service(int intr_alloc_flags);
void        gpio_uninstall_isr_service(void);
esp_err_t gpio_isr_handler_add(gpio_num_t gpio_num, gpio_isr_t isr_handler, void *args);
esp_err_t gpio_isr_handler_remove(gpio_num_t gpio_num);
```

Header File: [components/hal/include/hal/gpio_types.h](#)

Enum:

```

/*!< GPIO interrupt type */
typedef enum
{
    GPIO_INTR_DISABLE = 0, /*!< Disable GPIO interrupt */
    GPIO_INTR_POSEDGE = 1, /*!< GPIO interrupt type: rising edge */
    GPIO_INTR_NEGEDGE = 2, /*!< GPIO interrupt type: falling edge */
    GPIO_INTR_ANYEDGE = 3, /*!< GPIO interrupt type: both rising and falling edge */
    GPIO_INTR_LOW_LEVEL = 4, /*!< GPIO interrupt type: input low level trigger */
    GPIO_INTR_HIGH_LEVEL = 5, /*!< GPIO interrupt type: input high level trigger */
    GPIO_INTR_MAX,
} gpio_int_type_t;

```

```
void port_init (void);
```

```
void IRAM_ATTR gpio22_isr_handler(void *arg)
```

```
{
```

```
    tarea breve;
```

```
}
```

```
void IRAM_ATTR gpio23_isr_handler(void *arg)
```

```
{
```

```
    tarea breve;
```

```
}
```

```
void app_main(void)
```

```
{
```

```
    port_Init();
```

```
    //Instalar el gpio ISR
```

```
    gpio_install_isr_service(0);
```

```
    gpio_isr_handler_add(GPIO_NUM_22, gpio22_isr_handler, NULL);
```

```
    gpio_isr_handler_add(GPIO_NUM_23, gpio23_isr_handler, NULL);
```

```
    while (1)
```

```
    {
```

```
        vTaskDelay(pdMS_TO_TICKS(10));
```

```
    }
```

```
}
```




GRACIAS

